

[Marcar como hecha](#)

Proxy

El patrón de diseño Proxy tiene como objetivo el diseño de un objeto que sustituye a otro y controla su acceso.

Es un patrón de diseño **estructural** que te permite proporcionar un sustituto o marcador de posición para otro objeto. Un proxy controla el acceso al objeto original, permitiéndote hacer algo antes o después de que la solicitud llegue al objeto original.

Muy utilizado por ejemplo, cuando la clase es parte de una biblioteca cerrada de un tercero (componentes).

El patrón Proxy sugiere que crees una nueva clase proxy con la misma interfaz que un objeto de servicio original. Después actualizas tu aplicación para que pase el objeto proxy a todos los clientes del objeto original. Al recibir una solicitud de un cliente, el proxy crea un objeto de servicio real y le delega todo el trabajo.

Ya que implementa la misma interfaz que la clase original, puede pasarse a cualquier cliente que espere un objeto de servicio real.

Supongamos un negocio de coches online. Los vídeos de cada coche tardarían mucho en cargarse y transferirse. El patrón de diseño Proxy ofrece una solución a este problema difiriendo la creación de los sujetos hasta el momento en que el sistema los necesita realmente, en este caso tras un clic en la fotografía del vehículo, se vería el vídeo. El objeto fotografía se llama el proxy del vídeo sustituyendo la foto.

Conviene observar que los métodos del proxy tienen dos comportamientos posibles cuando el sujeto real no ha sido creado: o bien crean el sujeto real y a continuación le delegan el mensaje (es el caso del método clic del ejemplo), o bien ejecutan un código de sustitución (es el caso del método dibuja del ejemplo).

Los participantes del patrón de diseño Proxy son los siguientes:

- Sujeto (AnimacionInterfaz) es la interfaz común al proxy y al sujeto real.
- SujetoReal (Video) es el objeto que el proxy controla y representa.
- Proxy (AnimacionProxy) es el objeto que se sustituye por el sujeto real. Posee una interfaz idéntica a este último (interfaz Sujeto). Se encarga de crear y de destruir al sujeto real y de delegarle los mensajes.

Los proxies son muy útiles en programación orientada a objetos. Existen distintos tipos de proxy. Vamos a ilustrar tres:

- Proxy virtual: permite crear un objeto de tamaño importante en el momento adecuado.
- Proxy remoto: permite acceder a un objeto ejecutándose en otro entorno.
- Proxy de protección: permite asegurar el acceso a un objeto, por ejemplo mediante técnicas de autenticación.

Ejemplo de código

```
<?php
interface AnimacionInterfaz
{
    public function dibuja(): void;

    public function clic(): void;
}
```

El código fuente PHP de la clase Video que implementa esta interfaz aparece a continuación. En el marco de la simulación cada método muestra simplemente un mensaje en pantalla.

?

```
<?php

class Video implements AnimacionInterfaz
{
    public function clic(): void
    {
        echo 'Se ha recibido el evento clic' . PHP_EOL;
    }

    public function dibuja(): void
    {
        echo 'Visualizar el vídeo' . PHP_EOL;
    }

    public function carga(): void
    {
        echo 'Cargar el vídeo' . PHP_EOL;
    }

    public function reproduce(): void
    {
        echo 'Reproducir el vídeo' . PHP_EOL;
    }
}
```

El código fuente del proxy, y por tanto de la clase AnimacionProxy, aparece a continuación.

```
<?php

class AnimacionProxy implements AnimacionInterfaz
{
    protected Video $video;

    protected string $foto = 'Visualizar la foto';

    public function clic(): void
    {
        if (!isset($this->video)) {
            $this->video = new Video();
            $this->video->carga();
        }

        $this->video->reproduce();
    }

    public function dibuja(): void
    {
        if (isset($this->video)) {
            $this->video->dibuja();
        } else {
            $this->dibujaFoto($this->foto);
        }
    }

    public function dibujaFoto(string $foto): void
    {
        echo $foto . PHP_EOL;
    }
}
```

Por último, el código cliente se escribe de la siguiente manera.

```
<?php
include ('AnimacionInterfaz.php');
include ('AnimacionProxy.php');
include ('Video.php');

$animacion = new AnimacionProxy();
echo '<h3>->dibuja()</h3>';
$animacion->dibuja();
echo '<h3>->clic()</h3>';
$animacion->clic();
echo '<h3>->dibuja()</h3>';
$animacion->dibuja();
```

La ejecución de este programa muestra la diferencia de comportamiento del método dibuja del proxy según el método clic haya sido invocado previamente o no.

->dibuja()

Visualizar la foto

->clic()

Cargar el vídeo Reproducir el vídeo

->dibuja()

Visualizar el vídeo

Ejemplo segundo de código

```
<?php

/**
 * La interfaz Subject declara operaciones comunes tanto para el RealSubject como para el
 * Proxy. Mientras el cliente trabaje con RealSubject usando esta interfaz,
 * podrá pasarle un proxy en lugar de un sujeto real.
 */
interface Subject
{
    public function request(): void;
}

/**
 * El RealSubject contiene cierta lógica de negocio central. Normalmente, los RealSubjects son
 * capaces de hacer algún trabajo útil que también puede ser muy lento o delicado -
 * por ejemplo, corregir los datos de entrada. Un Proxy puede resolver estos problemas sin ningún
 * cambios en el código del RealSubject.
 */
class RealSubject implements Subject
{
    public function request(): void
    {
        echo "ObjetoReal: Manejando solicitud.<br>";
    }
}

/**
 * The Proxy tiene interface idéntica al RealSubject.
 */
class Proxy implements Subject
{
    ?
```

```

/**
 * @var RealSubject
 */
private $realSubject;

/**
 * El Proxy mantiene una referencia a un objeto de la clase RealSubject. Se
 * puede ser cargado de forma perezosa o pasado al Proxy por el cliente
 */
public function __construct(RealSubject $realSubject)
{
    $this->realSubject = $realSubject;
}

/**
 * Las aplicaciones más comunes del patrón Proxy son la carga perezosa,
 * el almacenamiento en caché, el control del acceso, el registro, etc. Un Proxy puede realizar una de
 * estas cosas y luego, dependiendo del resultado, pasar la ejecución al
 * mismo método en un objeto RealSubject vinculado.
 */
public function request(): void
{
    if ($this->checkAccess()) {
        $this->realSubject->request();
        $this->logAccess();
    }
}

private function checkAccess(): bool
{
    // aquí debería haber algunos chequeados reales
    echo "Proxy: Comprobación del acceso antes de lanzar una solicitud real.<br>";
    return true;
}

private function logAccess(): void
{
    echo "Proxy: Registro de la hora de la solicitud.<br>";
}
}

/**
 * El código del cliente debe trabajar con todos los objetos (tanto sujetos como
 * proxies) a través de la interfaz Subject para soportar tanto los sujetos reales como los
 * proxies. En la vida real, sin embargo, los clientes suelen trabajar con sus sujetos reales
 * directamente. En este caso, para implementar el patrón más fácilmente, puedes extender
 * tu proxy desde la clase del sujeto real.
 */
function clientCode(Subject $subject)
{
    $subject->request();
}

echo "Client: Ejecutar el código del cliente con un sujeto real:<br>";
$realSubject = new RealSubject();
clientCode($realSubject);

echo "<br>";

```

?

```
echo "Client: Ejecutando el mismo código de cliente con un proxy:<br>";  
$proxy = new Proxy($realSubject);  
clientCode($proxy);
```